

Lista de Exercícios.

O objetivo dessa lista é que vocês consigam desenvolver uma linha lógica para a solução de cada questão e após a solução, deve-se subir o código dessa questão **NO GITHUB**. Ou seja, ao responder à questão 1, você deve subir o código referente a essa questão no github. O link do repositório deve ser enviado neste formulário: [link](#)

A lista valerá **3.0 pontos**.

1 - Leia 2 valores inteiros (A e B). Após, o programa deve mostrar uma mensagem "Sao Multiplos" ou "Nao sao Multiplos", indicando se os valores lidos são múltiplos entre si.

Entrada

A entrada contém valores inteiros.

Saída

A saída deve conter uma das mensagens conforme descrito acima.

Exemplo de Entrada	Exemplo de Saída
6 24	Sao Multiplos
6 25	Nao sao Multiplos

2 - Leia 3 valores inteiros e ordene-os em ordem crescente. No final, mostre os valores em ordem crescente, uma linha em branco e em seguida, os valores na sequência como foram lidos.

Entrada

A entrada contém três números inteiros.

Saída

Imprima a saída conforme foi especificado.

Exemplo de Entrada	Exemplo de Saída
7 21 -14	-14 7 21 7 21 -14
-14 21 7	-14 7 21 -14 21 7

3 - Leia um valor inteiro **X** ($1 \leq X \leq 1000$). Em seguida mostre os ímpares de 1 até **X**, um valor por linha, inclusive o **X**, se for o caso.

Entrada

O arquivo de entrada contém 1 valor inteiro qualquer.

Saída

Imprima todos os valores ímpares de 1 até **X**, inclusive **X**, se for o caso.

4 - Escreva um programa que leia um valor inteiro N. Este N é a quantidade de linhas de saída que serão apresentadas na execução do programa.

Entrada

O arquivo de entrada contém um número inteiro positivo N.

Saída

Imprima a saída conforme o exemplo fornecido.

Exemplo de Entrada	Exemplo de Saída
7	1 2 3 PUM 5 6 7 PUM 9 10 11 PUM 13 14 15 PUM 17 18 19 PUM 21 22 23 PUM 25 26 27 PUM

5 - O novo prédio da Sociedade Brasileira de Computação (SBC) possui 3 andares. Em determinadas épocas do ano, os funcionários da SBC bebem muito café. Por conta disso, a presidência da SBC decidiu presentear os funcionários com uma nova máquina de expresso. Esta máquina deve ser instalada em um dos 3 andares, mas a instalação deve ser feita de forma que as pessoas não percam muito tempo subindo e descendo escadas.

Cada funcionário da SBC bebe 1 café expresso por dia. Ele precisa ir do andar onde trabalha até o andar onde está a máquina e voltar para seu posto de trabalho. Todo funcionário leva 1 minuto para subir ou descer um andar. Como a SBC se importa muito com a eficiência, ela quer posicionar a máquina de forma a minimizar o tempo total gasto subindo e descendo escadas.

Sua tarefa é ajudar a diretoria a posicionar a máquina de forma a minimizar o tempo total gasto pelos funcionários subindo e descendo escadas.

Entrada

A entrada consiste em 3 números, A_1 , A_2 , A_3 ($0 \leq A_1, A_2, A_3 \leq 1000$), um por linha, onde A_i representa o número de pessoas que trabalham no i -ésimo andar.

Saída

Seu programa deve imprimir uma única linha, contendo o número total de minutos a serem gastos com o melhor posicionamento possível da máquina.

Exemplos de Entrada	Exemplos de Saída
10 20 30	80
10 30 20	60
30 10 20	100

6 - Um Posto de combustíveis deseja determinar qual de seus produtos tem a preferência de seus clientes. Escreva um algoritmo para ler o tipo de combustível abastecido (codificado da seguinte forma: 1.Álcool 2.Gasolina 3.Diesel 4.Fim). Caso o usuário informe um código inválido (fora da faixa de 1 a 4) deve ser solicitado um novo código (até que seja válido). O programa será encerrado quando o código informado for o número 4.

Entrada

A entrada contém apenas valores inteiros e positivos.

Saída

Deve ser escrito a mensagem: "MUITO OBRIGADO" e a quantidade de clientes que abasteceram cada tipo de combustível, conforme exemplo.

Exemplo de Entrada	Exemplo de Saída
8 1 7 2 2 4	MUITO OBRIGADO Alcool: 1 Gasolina: 2 Diesel: 0

7 - Dado um inteiro n , retorne uma lista de strings **answer** (indexado em

1) onde:

- `answer[i] == "FizzBuzz"` se `i` for divisível por 3 e por 5.
- `answer[i] == "Fizz"` se `i` for divisível por 3.
- `answer[i] == "Buzz"` se `i` for divisível por 5.
- `answer[i] == i` (como uma string) se nenhuma das condições acima for verdadeira.

Exemplo 1:

- **Entrada:** `n = 3`
- **Saída:** `["1","2","Fizz"]`

Exemplo 2:

- **Entrada:** `n = 5`
- **Saída:** `["1","2","Fizz","4","Buzz"]`

Exemplo 3:

- **Entrada:** `n = 15`
- **Saída:**
`["1","2","Fizz","4","Buzz","Fizz","7","8","Fizz","Buzz","11","Fizz","13","14","FizzBuzz"]`

8 - O Brasil é o país sede da copa esse ano. Porém, há muitas pessoas protestando contra o governo. Em redes sociais é possível ver pessoas afirmando que não vai ter copa devido aos protestos.

Mas esses rumores de que não haverá copa são totalmente falsos, a presidente Dilma Rousseff já avisou: vai ter copa sim, e se reclamar vai ter duas!

Entrada

A entrada contém vários casos de teste e termina com EOF. Cada caso de teste consiste de uma linha contendo o número **N** de reclamações sobre a copa encaminhadas para a presidente ($0 \leq \mathbf{N} \leq 100$).

Saída

Para cada teste, a saída consiste de uma linha dizendo "vai ter copa!" caso não haja reclamações para a presidente. Caso haja reclamações, a saída deverá dizer "vai ter duas!".

Exemplo de Entrada	Exemplo de Saída
0	vai ter copa!
1	vai ter duas!
0	vai ter copa!
2	vai ter duas!
100	vai ter duas!
0	vai ter copa!

9 - A sequência de números 0 1 1 2 3 5 8 13 21... é conhecida como série de Fibonacci. Nessa sequência, cada número, depois dos 2 primeiros, é igual à soma dos 2 anteriores. Escreva um algoritmo que leia um inteiro N ($N < 46$) e mostre os N primeiros números dessa série.

Entrada

O arquivo de entrada contém um valor inteiro N ($0 < N < 46$).

Saída

Os valores devem ser mostrados na mesma linha, separados por um espaço em branco. Não deve haver espaço após o último valor.

Exemplo de Entrada	Exemplo de Saída
5	0 1 1 2 3

10 - Dada uma configuração inicial, a máquina de criptografia alemã Enigma, da Segunda Guerra Mundial, substituía cada letra digitada no teclado por outra letra. A substituição era bastante complexa, mas a máquina tinha uma vulnerabilidade: uma letra nunca seria substituída por ela mesma! Essa vulnerabilidade foi explorada por Alan Turing, que trabalhou na criptoanálise da Enigma durante a guerra. O objetivo era encontrar a configuração inicial da máquina usando a suposição de que

a mensagem continha uma certa expressão usual da comunicação, como por exemplo a palavra **ARMADA**. Essas expressões eram chamadas de cribs. Se a mensagem cifrada era, por exemplo, **FDMLCRDMRALF**, o trabalho de testar as possíveis configurações da máquina era simplificado porque a palavra **ARMADA**, se estivesse nessa mensagem cifrada, só poderia estar em duas posições, ilustradas na tabela abaixo com uma seta. As demais cinco posições não poderiam corresponder ao crib **ARMADA** porque ao menos uma letra do crib, sublinhada na tabela abaixo, casa com sua correspondente na mensagem cifrada; como a Enigma nunca substituiria uma letra por ela própria, essas cinco posições poderiam ser descartadas nos testes.

F	D	M	L	C	R	D	M	R	A	L	F
A	R	<u>M</u>	A	D	A						
	A	R	M	A	D	A	←				
		A	R	M	A	<u>D</u>	A				
			A	R	M	A	D	A	←		
				A	<u>R</u>	M	A	D	<u>A</u>		
					A	R	<u>M</u>	A	D	A	
						A	R	M	<u>A</u>	D	A

Neste problema, dada uma mensagem cifrada e um crib, seu programa deve computar o número de posições possíveis para o crib na mensagem cifrada.

Entrada

A primeira linha da entrada contém a mensagem cifrada, que é uma sequência de pelo menos uma letra e no máximo 104 letras. A segunda linha da entrada contém o crib, que é uma sequência de pelo menos uma letra e no máximo o mesmo número de letras da mensagem. Apenas as 26 letras **MAIUSCULAS**, sem acentuação, aparecem na mensagem e no crib.

Saída

Imprima uma linha contendo um inteiro, indicando o número de posições possíveis para o crib na mensagem cifrada.

Exemplo de entrada 1 FDMLCRDMRALF ARMADA	Exemplo de saída 1 2
Exemplo de entrada 2 AAAAABABABABABABABABA ABA	Exemplo de saída 2 7